Somewhat counterintuitively, Numeric does define a couple of division methods. It defines self.div(x) as integer division self/x using the self objects definition of /. If you want a float result, then fdiv does the same division but returns a float. The method quo does the same division, but returns a rational if the argument is an integer or rational, or a float if the argument is a float. The method divmod returns a two-element array such that self.divmod(x) = [self/x.floor, self % x]. The mod value is also available as remainder.

Any number can be converted to an integer with to_int, though the kernel method Integer is probably preferred, and you can go to a complex number with to_c. Although it makes the most sense for rationals, numerator and denominator are defined in Numeric.

Numeric defines a set of query methods that are valid for all number types. The set positive?, negative?, nonzero?, and zero? all return true or false based exactly on the check in the name of the method. It's fairly common to see x.zero? rather than x == 0 as a condition. The infinite? method checks against the -Infinity or +Infinity special values; the inverse method is finite?. If the number is an Integer?, then integer? returns true, and similarly real?. Note that the check is based on the type of the number, not the value; 1.0.integer? is false.

You can convert any number to a collection with the step method, which takes a variety of arguments and a block. It can take two positional arguments—to and by—in which case the block is called with self as an argument. The by amount is added to self and the block is called again until the to argument is crossed. If by is positive, it crosses by going higher; if by is negative, it crosses by going lower. So it's roughly equivalent to this implementation:

```
def step(to = nil, by = 1, &block)
  counter = self
  start_relationship = self <=> to
  while (counter <=> to) == start_relationship
    block.call(counter)
    counter += by
  end
end
```

As you can see, to defaults to nil, meaning an infinite sequence, and by defaults to 1. You can also specify to: and by: as keyword arguments and that will also work. If no block is given, the real method, unlike our scratch implementation earlier, will return an Enumerator that will successively return the values that would have been passed to the block.

Integer Class

The Integer class inherits from the Numeric class and implements a number of the methods described earlier on its own for memory or speed performance purposes. It also adds some additional functionality of its own.

In addition to implementing the regular arithmetic operators, +, -, *, / (which is integer division and truncates the result), % (also available as modulo), and ** (also available as pow), Integer also implements bitwise arithmetic.

The bitwise AND (&) operator returns a new number with a 1 in each bit that is 1 in the binary representation of both operands. The bitwise OR (|) operator returns a new number with a 1 in the binary operation of either operand. The bitwise XOR (^) operator returns a new number with a 1 in each bit that is 1 in the binary representation of one but not both operands. The << shifts the bits of the left operand the number in the right operand, and ~